



BSc (Hons) in **Information Technology Specialized in AI**
IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:
`'grid_size': (self.width, self.height)`

```
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent` .
2. Import required structures: `from collections import deque` and `import heapq` .
3. **BFS**: Create `def bfs_search(...)` . Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS**: Create `def dfs_search(...)` . Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS**: Create `def ucs_search(...)` . Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__` , add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'` .
2. In `sense_and_act(self, percept)` , check if `self.plan` is empty.

3. If empty, find the closest food pellet from `percept[ 'all_food' ]` , execute the search method matching `self.active_algo` , and store the resulting sequence of actions in `self.plan` .
4. Return the first action from the plan using `return self.plan.pop(0)` .
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

The State Space is the set of all possible configurations or states of the environment, together with the actions that connect those states. It is a fixed structure defined by the problem, and the same state exists only once in the state space even if there are multiple ways to reach it.

The Search Tree, on the other hand, is created during the search process. It represents the different paths that the search algorithm explores from the initial state. The same state can appear multiple times in the search tree if it is reached through different paths. Therefore, the state space can be thought of as the complete map of possible states, while the search tree represents the paths explored by the search algorithm.

2. (Understand) In Step 1.2, you were instructed to use a ``reached`` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set prevents the search algorithm from repeatedly expanding states that it has already visited. Without a reached set, the algorithm may treat the same state as a new state every time it is reached through a different path.

This is especially important in a grid because the same cell can often be reached through multiple paths. For example, a cell could be reached by moving Right and then Up, or Up and then Right. Without the reached set, the algorithm would unnecessarily explore these repeated states.

For DFS, this can be even more serious because the algorithm could move between two neighbouring cells repeatedly, such as Right, Left, Right, Left, without knowing that these states have already been visited. Therefore, the reached set helps prevent repeated exploration and infinite loops, making the search more efficient.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS is optimal in this environment because it explores the search space level by level. It first explores all states that can be reached in one step, then all states that can be reached in two steps, and continues in this way. Therefore, when BFS reaches the goal for the first time, it has found the shortest path to that goal.

This is true because all movements in the grid have the same cost. Therefore, minimizing the number of steps also minimizes the total path cost.

DFS is not guaranteed to find the shortest path because it explores one direction as deeply as possible before backtracking. It does not compare all possible paths based on their length. As a result, it can find a longer and more winding path before finding the shorter path. This explains why BFS/UCS can find a shorter path while DFS may take a longer route on the same grid.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus

DFS.

BFS stores a large number of nodes in its frontier at each level. Therefore, on a 1000×1000 grid, the number of states stored in memory can become very large, making memory usage a major limitation.

DFS mainly stores the current path and the nodes needed for backtracking, so it generally requires less frontier memory than BFS. However, when DFS uses a reached set, it also needs memory to store the states that have already been visited. On a very large grid, this set can also become large.

Therefore, BFS is generally more likely to face a serious memory bottleneck because it needs to maintain a large frontier, while DFS uses less frontier memory but may produce longer, winding paths and does not guarantee an optimal solution.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation



FACULTY OF COMPUTING